

ORACLE

Cloud Operations for ExaDB-D

Operations with Ansible

Operations Advisory Team

Office of CTO | EMEA Solution Engineering

July, 2026

Agenda

- Introduction
- Terraform & Ansible
- Provisioning Operations Scenarios
- Change Operations Scenarios
- Maintenance Operations Scenarios



Introduction

Introduction



- This Workshop is intended to give some detailed information about how Terraform and Ansible can be used to provision ExaDB-D (formerly ExaCS) resources and perform day 2 operations.
- We will focus on where it is recommended to set the border between the two IaC technologies to take the best from both
- We'll review the existing OCI Ansible collection and see how to extend it to cover your own needs

Terraform & Ansible

What is and does Terraform

Terraform is a **declarative** language that defines the **desired state** of infrastructure.

It uses a **state file** to consistently provision **Landing Zones (LZ)** and **workload** resources across **any cloud**.

Terraform is great to **provision the Landing Zone and workload** resources.



Declarative Language

Defines the "**what**", not the "how". A simple configuration expresses the entire desired state.



State Management

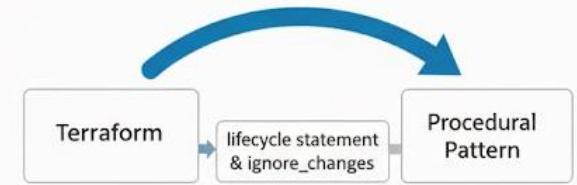
A **state file** acts as the **single source of truth**, mapping configurations to real-world resources.



Landing Zones Workload Resources

LZ set the **foundation** and **guardrails**.
Workload resources add the stacks on top across **any cloud**.

Terraform is not enough



The Limits

Terraform has some boundaries. It cannot manage tasks inside a resource, only the resource itself.

-  **No patching:** Not for OS/DB/app patching or upgrades
-  **No in-guest config:** Doesn't install packages, edit files, or manage services on existing hosts
-  **No app/data config:** doesn't manage DB schemas/users or app settings.
-  **Non-Updatable** changes (needs to re-deploy)
-  **Last resort only:** Provisioners (remote-exec/local-exec) and generic helpers (terraform_data, deprecated null_resource) aren't suited for Config Management and can leak secrets to state/logs

The Solution

Use the lifecycle statement block to **bridge the gap** between **declarative IaC** and **procedural** realities.

```
lifecycle {
  ignore_changes = [
    database[0].admin_password,      <--- Ignoring by default
    database[0].database_software_image_id, <--- To ignore against db_home patching
    database_software_image_id,      <--- To ignore against db_home patching
    db_version,                      <--- To ignore against db_home patching
  ]
}
```

The ignore_changes skip diffs for specific fields (config edits or drift)

What is Ansible

Ansible is a declarative, **YAML-based** automation engine, **agentless** IaC tool used primarily for **configuration management**.

Agent less

No permanent **agents** on managed systems.
Uses **temporary** network **connections** via
SSH or Windows Remote Management

Vs. Alternatives

Chef, Puppet and Salt require installed agents.
Ansible eliminates this **complexity** and overhead

Minimum Requirement

Only requires Python installed on managed systems.
Nothing more

Central Orchestration

Centralized and efficient **architecture**
All **orchestration** is done from a **control node** (masterd host).

SSH Security

Leverages native **SSH security** without additional ports or running daemons

Zero Overhead

Zero resource consumption on managed servers when no tasks are running

Playbooks: The Heart of Ansible

Playbooks are the language by which Ansible orchestrates, configures, administers, or deploys systems.

Execution flow:

Playbooks → Roles → Tasks → Hosts

GIT Versioning

Playbooks can be stored and versioned in GIT (SSoT), allowing full control of code

Code Reusability

Import playbooks (collections) for shared common code across teams and projects

CI/CD Integration

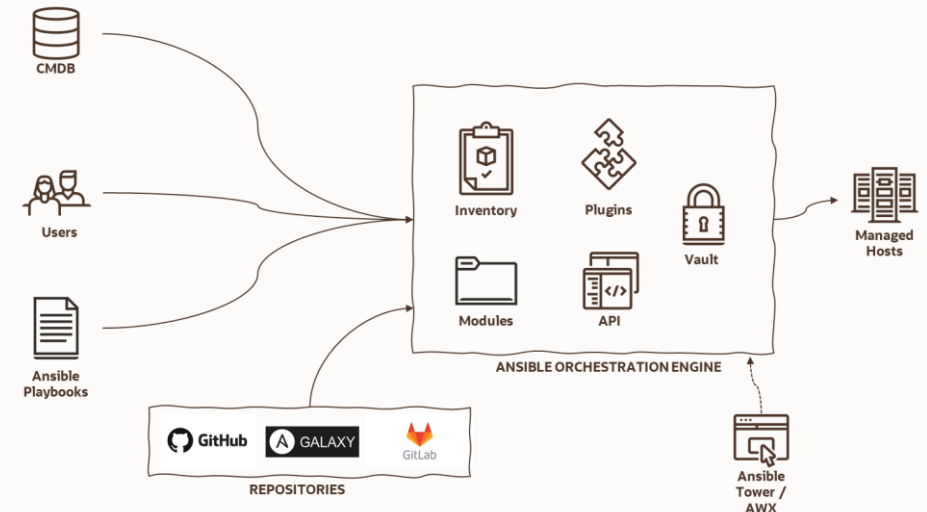
TERRAFORM

JENKINS

GITLAB

OTHER

Ansible can be used as an executor in CI/CD runners, integrating seamlessly with tools like Terraform, Jenkins and more



OCI Ansible collection

Ansible Collection helps you **automate the provisioning and configuring** of Oracle Cloud Infrastructure services and resources

Github repository

[OCI Ansible collection](#)

Ansible Tower/ AWX Integration

[Ansible Tower](#)

[AWX.](#)

Ansible Inventory Plugin

[Static Inventory](#) & [Dynamic inventory](#)

Supported Services

The Oracle Cloud Infrastructure Ansible Collection provides **hundreds of modules** to **automate** provisioning and configuration of **OCI resources**.

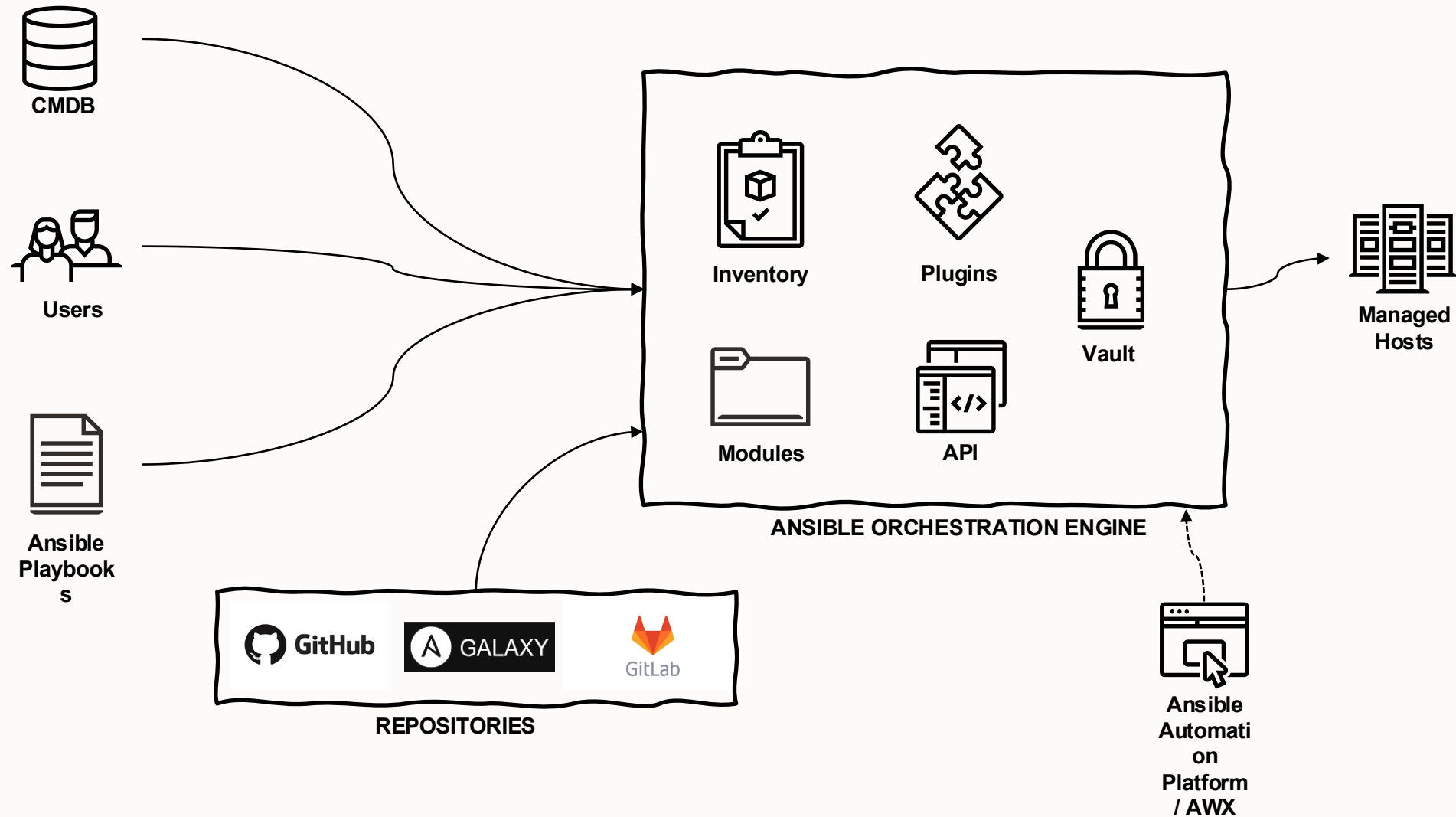
Minimal Requirements

The OCI **Ansible collection** requires Ansible version. **2.9 or later**

OCI Collection key benefits

Provision & configure OCI via Ansible.
Dynamic inventory for discovery.
Centralized runs (AWX/Tower, OLAM).
Broad, Oracle-maintained coverage.

Ansible Architecture



Using OCI Ansible Collection | Using the collection

- **Install**

```
$ ansible-galaxy collection install oracle.oci
```

- **Update**

```
$ ansible-galaxy collection install --force oracle.oci
```

- **Test installation**

```
$ ansible localhost -m oracle.oci.oci_object_storage_namespace_facts
```

- **Get documentation**

```
$ ansible-doc oracle.oci.oci_object_storage_bucket_facts  
$ ansible-doc -t inventory oracle.oci.oci
```

- **Setup authentication**

```
Use same method as OCI CLI (.oci/config)
```

Using OCI Ansible Collection | Execution Environments

- Package the collection + OCI Python SDK in a container image (Execution Environment) so every CI/CD runner and AAP/AWX job uses identical, versioned dependencies
- Build the image with ansible-builder and test locally with ansible-navigator — no more “works on my machine” caused by SDK version drift

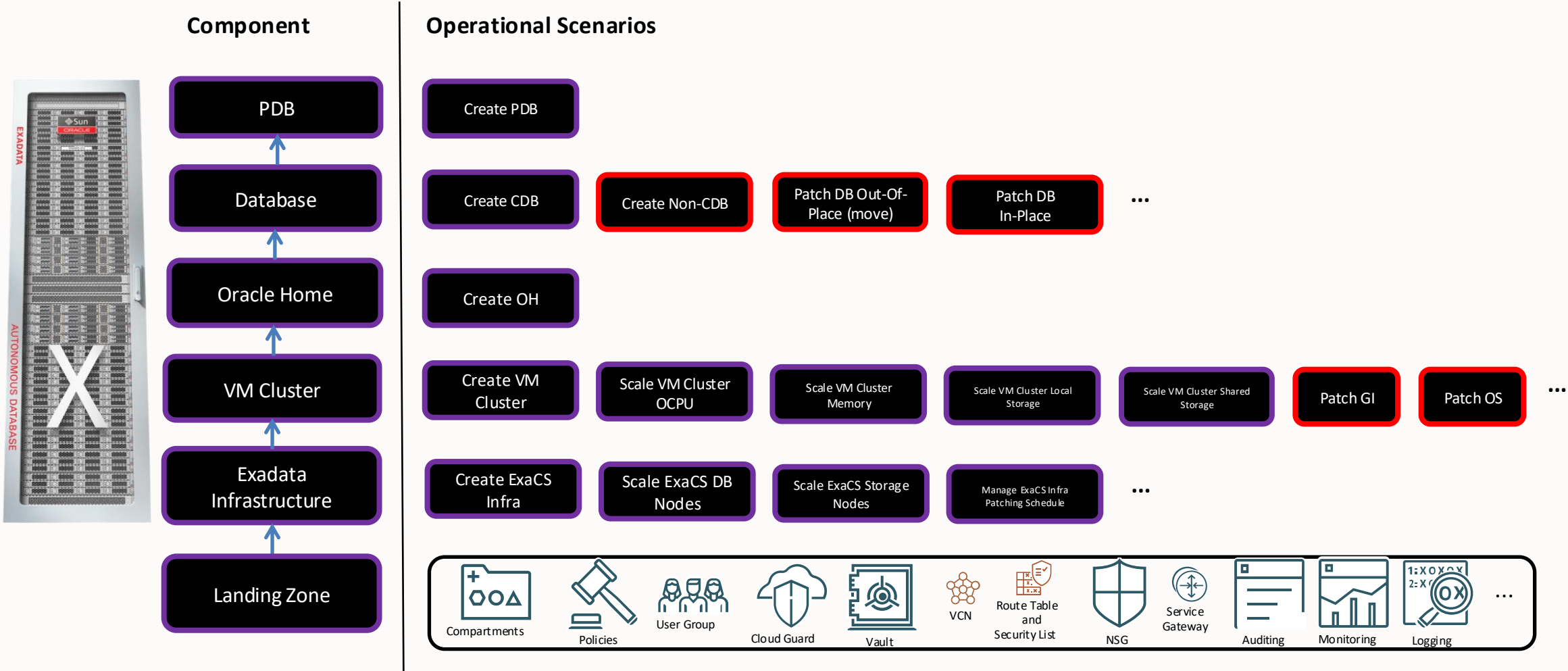
```
# execution-environment.yml (ansible-builder v3)
version: 3
images:
  base_image:
    name: quay.io/ansible/community-ee-base:latest
dependencies:
  galaxy:
    collections:
      - name: oracle.oci
        version: 5.5.0
  python:
    - oci>=2.152.1

# $ ansible-builder build -t oci-ee:5.5.0
# $ ansible-navigator run getFacts.yml --eei oci-ee:5.5.0
```

Operational Scenarios: Terraform + Ansible extension

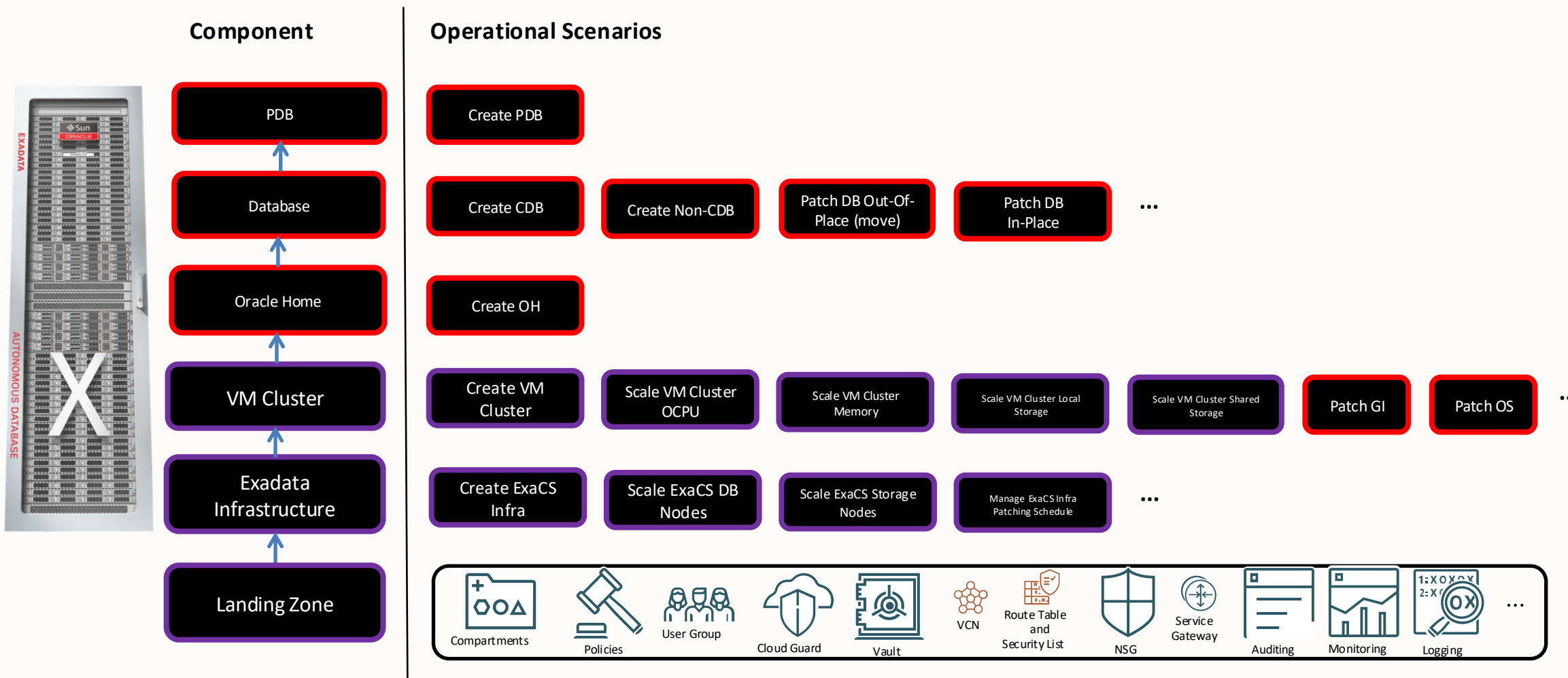
Ansible managed

Terraform managed



Terraform + Ansible extension (recommended approach)

 Ansible managed
 Terraform managed

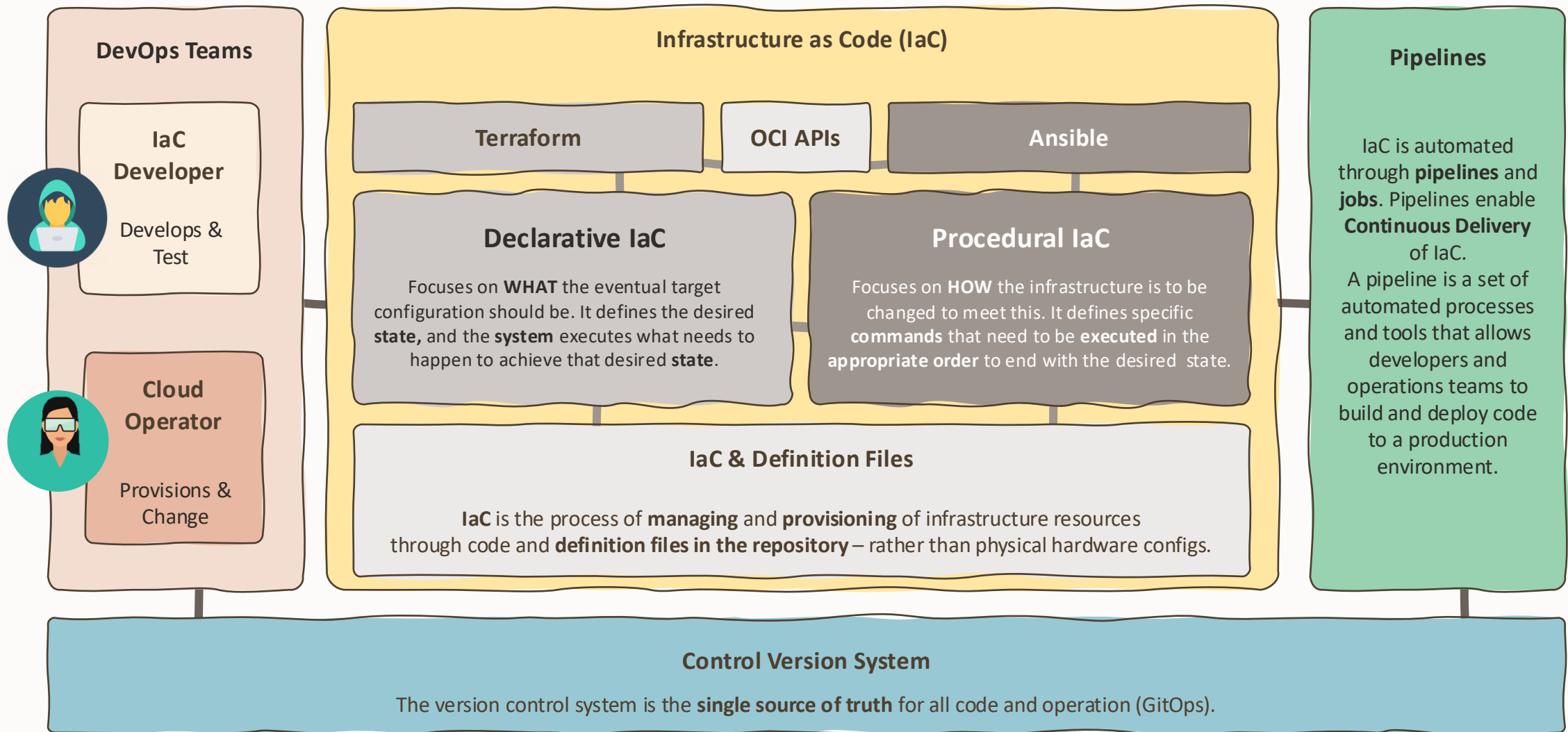


What is GitOps?

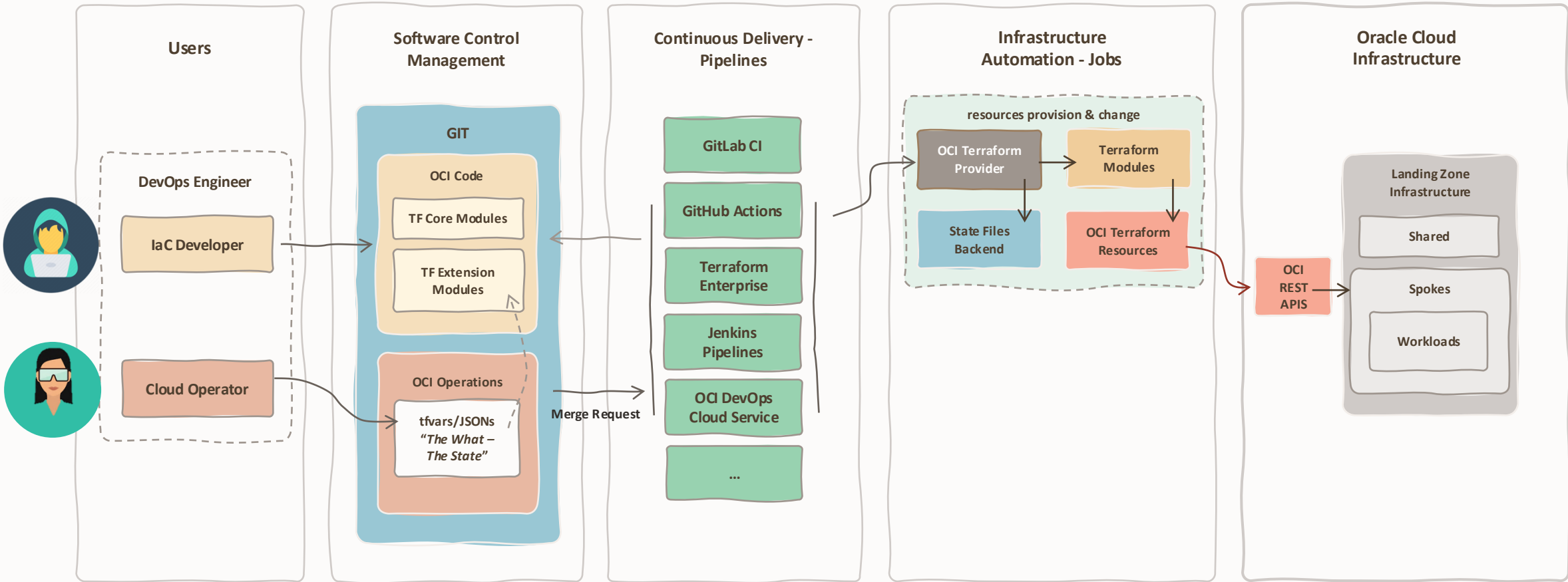
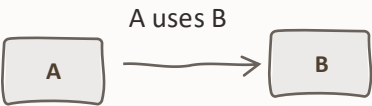
*GitOps is an operational practice that **applies DevOps best practices**, like **version control, collaboration, compliance**, and **CI/CD** to infrastructure automation.*

GitOps becomes a great common multi-cloud operating model.

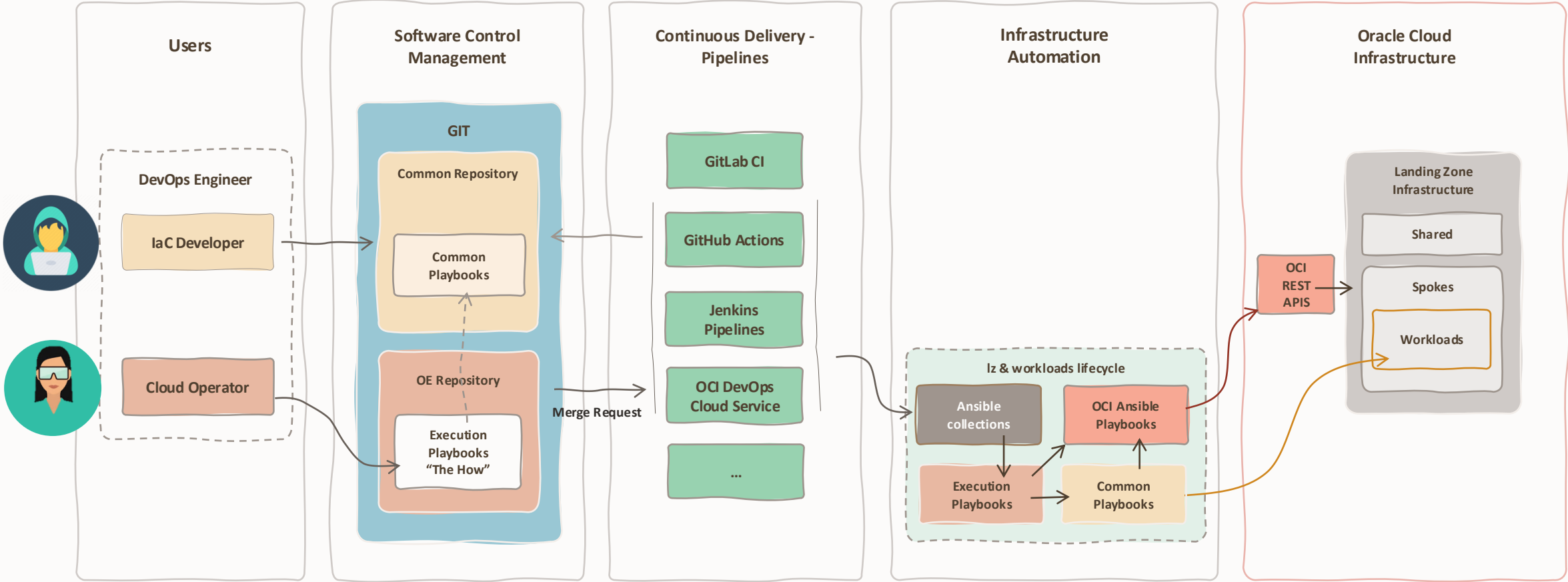
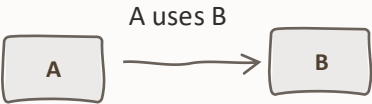
IaC Technology Context & Personas



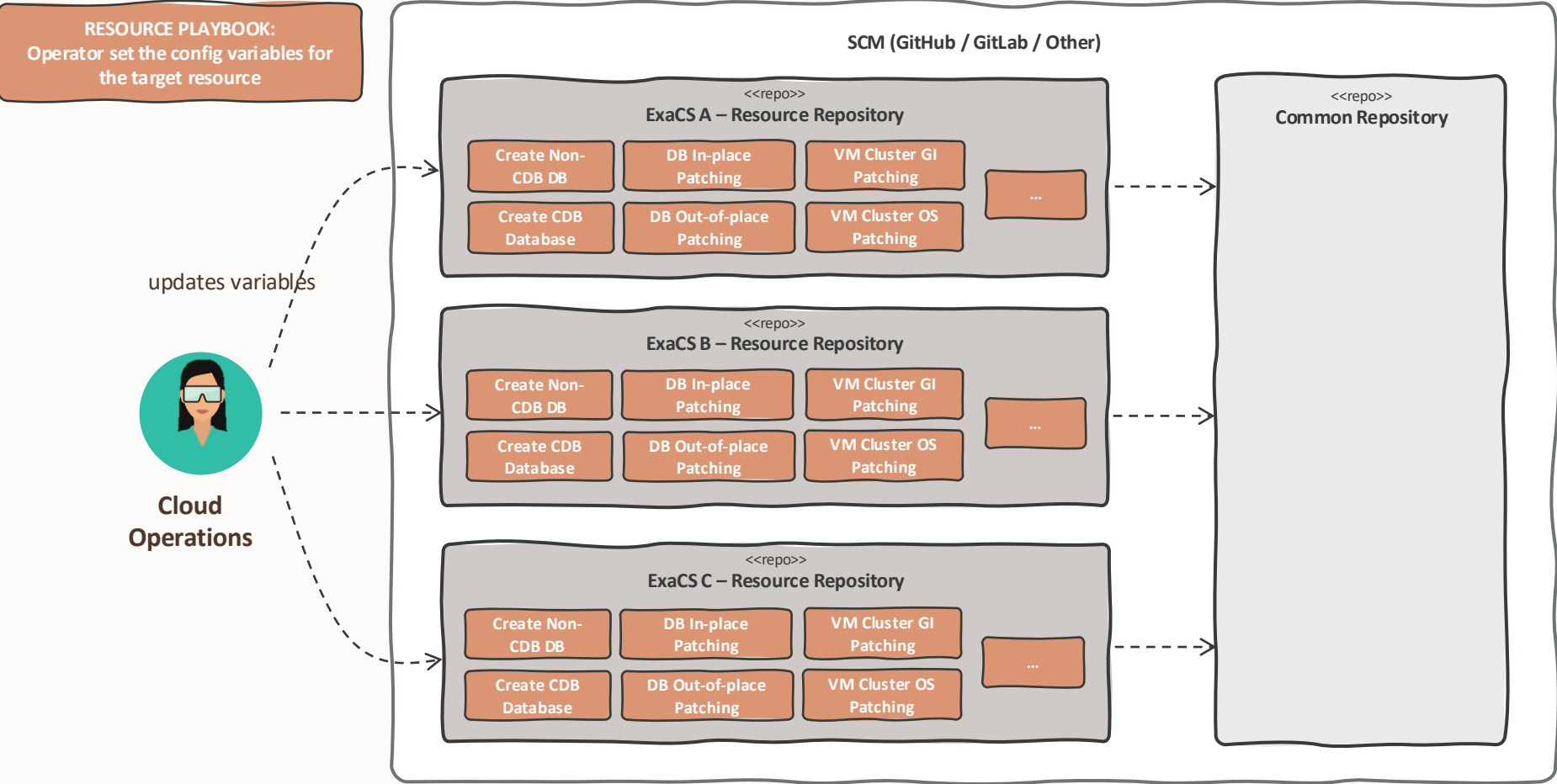
GitOps using a Declarative Pattern with Terraform



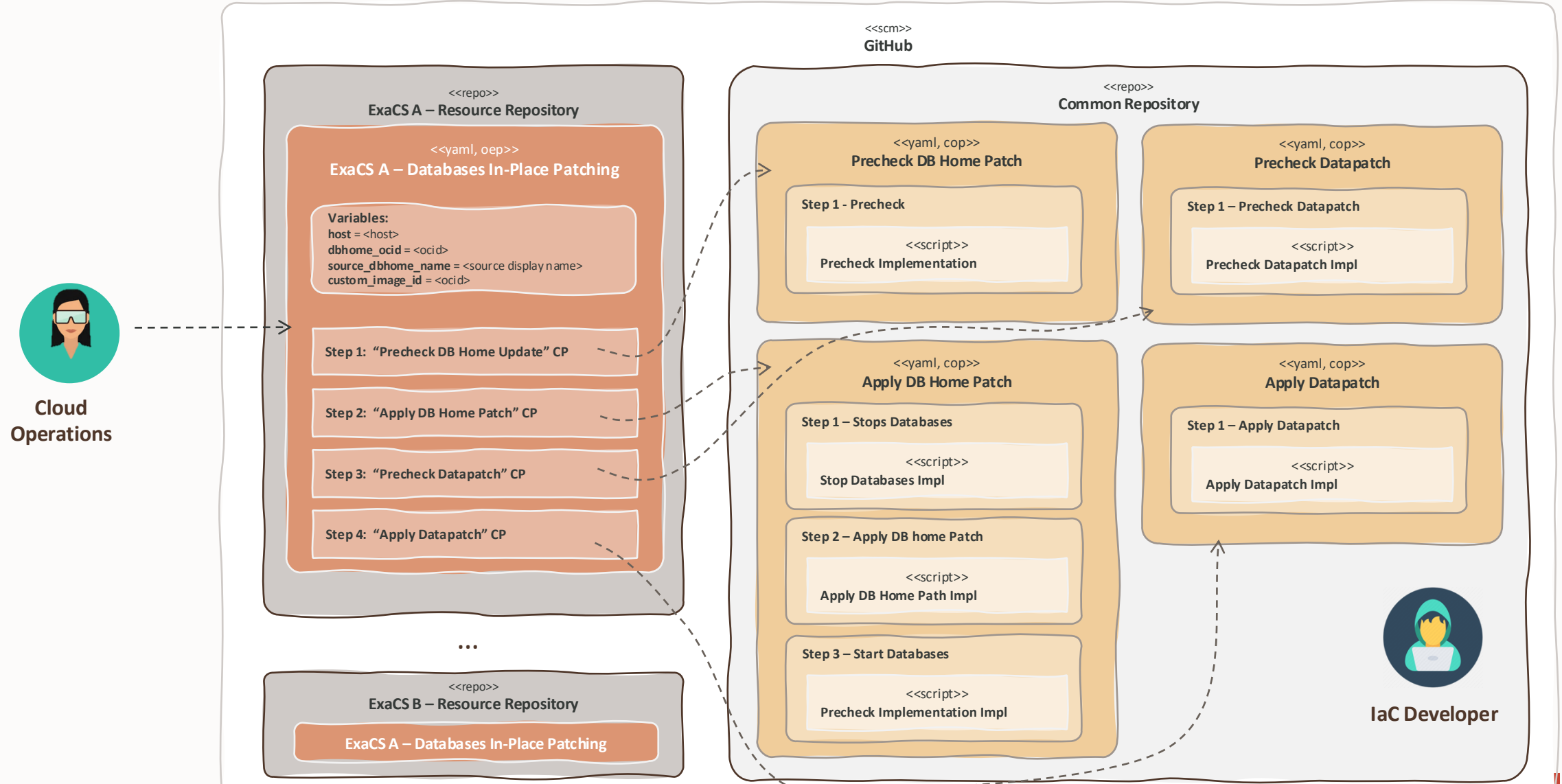
GitOps using a Procedural Pattern with Ansible



GitOps | Resource Repositories content is driven by Operation Scenarios



ExaCS | In-Place Patching | Repository View



Using OCI Ansible Collection | The ansible configuration file

- **Example ansible configuration file:**

```
[defaults]
private_key_file = ~/.ssh/<your private SSH key>
remote_user = opc
stdout_callback = debug
host_key_checking = False

[persistent_connection]
command_timeout = 3600

#[inventory]
enable_plugins = oracle.oci.oci
```

Using OCI Ansible Collection | Managing credentials securely

- Never hardcode passwords in playbooks or Git repos (the dummy values in this deck are for readability only). Mask sensitive output with `no_log: true`
- Encrypt static variable files with `ansible-vault`, or better: retrieve secrets at runtime from OCI Vault with the collection — nothing sensitive at rest

```
- name: Get admin password from OCI Vault (Secrets service)
  oracle.oci.oci_secrets_secret_bundle_facts:
    secret_id: "ocid1.vaultsecret.oc1..."
  register: bundle

- name: Decode secret into a variable (never logged)
  ansible.builtin.set_fact:
    admin_password: "{{ bundle.secret_bundle.secret_bundle_content.content | b64decode }}"
  no_log: true

# Alternative (static files): ansible-vault encrypt group_vars/all/vault.yml
#   and pass --vault-password-file to ansible-playbook in the CI/CD runner
```

Using OCI Ansible Collection | Using the inventory

- **Example inventory definition for ExaCS:**

```
# DB Hosts
plugin: oracle.oci.oci

config_file: ~/.oci/config
config_profile: DEFAULT

regions:
  - eu-frankfurt-1

enable_parallel_processing: yes

fetch_db_hosts: True
fetch_compute_hosts: False

hostname_format_preferences:
  - "hostname+'.sub12170906420.vcnexacs.oraclevcn.com'"

compartments:
  - compartment_ocid: ocid1.compartment.oc1...

#debug: true
```


Using OCI Ansible Collection | Using the inventory

- **Example output:**

```
$ ansible-inventory -i dbhosts.oci.yml --list
{
  (...
  },
  "all": {
    "children": [
      "Shared_Specialists",
      "all_hosts",
      "bWnP_EU-FRANKFURT-1-AD-3",
      "db_hosts",
      "region_eu-frankfurt-1",
      "ungrouped"
    ]
  },
  "all_hosts": {
    "hosts": [
      "emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com",
      "emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com"
    ]
  },
  "bWnP_EU-FRANKFURT-1-AD-3": {
    "hosts": [
      "emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com",
      "emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com"
    ]
  },
  "db_hosts": {
    "hosts": [
      "emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com",
      "emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com"
    ]
  },
  "region_eu-frankfurt-1": {
    "hosts": [
      "emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com",
      "emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com"
    ]
  }
}
```

Using OCI Ansible Collection | Using the inventory

- **Example playbook:**

```
- hosts: all
  tasks:
    - name: Ansible ping
      ansible.builtin.ping:
```

- **Using the playbook with the inventory elements:**

```
$ ansible-playbook -i dbhosts.oci.yml ping.yml
```

- **Example output:**

```
PLAY [all]
*****

TASK [Gathering Facts]
*****
ok: [emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com]
ok: [emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com]

TASK [Ansible ping]
*****
ok: [emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com]
ok: [emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com]

PLAY RECAP *****
emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

Using OCI Ansible Collection | Using the inventory

- **Example playbook:**

```
- hosts: all
  tasks:
    - name: Ansible ping
      ansible.builtin.ping:
```

- **Using the playbook with the inventory elements:**

```
$ ansible-playbook -i dbhosts.oci.yml ping.yml
```

- **Example output:**

```
PLAY [all]
*****

TASK [Gathering Facts]
*****
ok: [emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com]
ok: [emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com]

TASK [Ansible ping]
*****
ok: [emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com]
ok: [emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com]

PLAY RECAP *****
emeaexacsvm1-1bcej1.sub12170906420.vcnexacs.oraclevcn.com : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
emeaexacsvm1-1bcej2.sub12170906420.vcnexacs.oraclevcn.com : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

Using OCI Ansible Collection | Using the collection

- **Example playbook:**

```
---
- name: List summary of existing buckets in OCI object storage
  collections:
    - oracle.oci
  connection: local
  hosts: localhost
  vars:
    compartment_ocid: 'ocid1.compartment.oc1..aaaaaaaay4x2z23bz1up7o6gmksl5sbhxc2st5ifq6sg4hxpdulvcu7m7a'
    tenancy_namespace: 'frlzmy5hrkeg'
  tasks:
    - name: List bucket facts
      oci_object_storage_bucket_facts:
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-frankfurt-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
      loop: "{{ result.buckets }}"
```

- **Execution:**

```
$ ansible-playbook list_buckets.yml
```

Using OCI Ansible Collection | Using the collection

- Example output:

```
PLAY [List summary of existing buckets in OCI object storage]
*****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [List bucket facts]
*****
ok: [localhost]

TASK [Dump result] *****
ok: [localhost] => (item={'namespace': 'frlzmy5hrkeg', 'name': 'anewson', 'compartment_id': 'ocid1.compartment.oc1..aaaaaaaay4x2z23bz1up7o6gmks15sbhxc2st5ifq6sg4hxpdulvcu7m7a',
'created_by': 'ocid1.user.oc1..aaaaaaai2teciej4mzt625xop6zoxturno7bgr62jzase2w5hsmarcp4beq', 'time_created': '2023-02-16T10:02:04.312000+00:00', 'etag': 'e9283e0c-4d34-4c22-97d7-
a5cd5500f15c', 'freeform_tags': None, 'defined_tags': None}) => {}

MSG:

Bucket name: anewson
ok: [localhost] => (item={'namespace': 'frlzmy5hrkeg', 'name': 'aputhenc', 'compartment_id': 'ocid1.compartment.oc1..aaaaaaaay4x2z23bz1up7o6gmks15sbhxc2st5ifq6sg4hxpdulvcu7m7a',
'created_by': 'ocid1.user.oc1..aaaaaaa52pdqugkjm5x4amn4veakiimplm4htkktbtbvboxjmfthbc2xuhga', 'time_created': '2023-02-20T13:42:22.416000+00:00', 'etag': '58596e14-ea18-497c-8238-
cfbbc9c93e72', 'freeform_tags': None, 'defined_tags': None}) => {}

MSG:

Bucket name: aputhenc
ok: [localhost] => (item={'namespace': 'frlzmy5hrkeg', 'name': 'prodbuck', 'compartment_id': 'ocid1.compartment.oc1..aaaaaaaay4x2z23bz1up7o6gmks15sbhxc2st5ifq6sg4hxpdulvcu7m7a',
'created_by': 'ocid1.user.oc1..aaaaaaaadz4gzkxr7axe34kgldqi3rmpnhyyxcnqghe7343s5fmc4uobtxq', 'time_created': '2022-05-30T05:20:16.929000+00:00', 'etag': '8f667a24-8f2f-4c1b-b2e6-
77fdb023a0e1', 'freeform_tags': None, 'defined_tags': None}) => {}

MSG:

Bucket name: prodbuck

PLAY RECAP *****
localhost                : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Using OCI Ansible Collection | Using the collection

Overruling modules arguments

- **Example playbook:**

```
---
- name: List summary of existing buckets in OCI object storage
  collections:
    - oracle.oci
  connection: local
  hosts: localhost
  vars:
    compartment_ocid: 'ocid1.compartment.oc1...'
    tenancy_namespace: 'frlzmy5hrkeg'
  tasks:
    - name: List bucket in Frankfurt region
      oci_object_storage_bucket_facts:
        config_profile_name: "DEFAULT"
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-frankfurt-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
        loop: "{{ result.buckets }}"

    - name: List bucket in Amsterdam region
      oci_object_storage_bucket_facts:
        config_profile_name: "DEFAULT"
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-amsterdam-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
        loop: "{{ result.buckets }}"
```

Using OCI Ansible Collection to gather ExaCS components information

PROPOSED DEMO

❑ 1) Execution playbook for getting ExaCS Infrastructure facts:

- OCI config file (~/.oci/config):
- ansible.cfg
- common playbooks directory

❑ 2) Main execution playbook (getFacts.yml)

```
---
- name: ExaCS get facts example
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    compartment_id: "ocid1.compartment..."
    cloud_exadata_infrastructure_ocid: "ocid1.cloudexadatainfrastructure..."

  tasks:
    - import_tasks: common/getExaCSInfra.yml
      vars:
        compartment_ocid: "{{ compartment_id }}"
        cloud_exadata_infrastructure_ocid: "{{ cloud_exadata_infrastructure_ocid }}"
```

Using OCI Ansible Collection to gather ExaCS components information

PROPOSED DEMO

❑ 3) Common operation playbook (getExaCSInfra.yml)

```
---
- name: Get a specific cloud_exadata_infrastructure
  oci_database_cloud_exadata_infrastructure_facts:
    # required
    cloud_exadata_infrastructure_id: "{{ cloud_exadata_infrastructure_ocid }}"
  register: exainfra
- debug:
  msg: "{{ exainfra }}"

- name: List cloud_exadata_infrastructures
  oci_database_cloud_exadata_infrastructure_facts:
    # required
    compartment_id: "{{ compartment_ocid }}"

    # optional
    sort_by: TIMECREATED
    sort_order: ASC
    lifecycle_state: AVAILABLE
  register: result
- debug:
  msg: "{{ result }}"
```

❑ 4) Playbook execution

```
$ ansible-playbook getFacts.yml
```


Provisioning Operations Scenarios

Provisioning Operations Scenarios

- In this section we'll talk about the following Provisioning Operations Scenarios:
 - Create Database Software Images
 - Provision Oracle Homes
 - Provision Container Databases (CDBs)
 - Provision Non Container Databases (Non-CDBs)
 - Provision Pluggable Databases (PDBs)
 - Provision Data Guard Association reusing existing VM Cluster
 - Provision Data Guard Association creating a new VM Cluster
 - Delete Database Software Images
 - Delete Oracle Homes
 - Delete Container Databases (CDBs)
 - Delete Pluggable Databases (PDBs)
- *For simplicity, we'll use just examples of using the different OCI Ansible Collection Modules without Common Operation Playbook. All complete playbooks are available in the workshop companion repository*

Using OCI Ansible Collection | Using the collection

Overruling modules arguments

- **Example playbook:**

```
---
- name: List summary of existing buckets in OCI object storage
  collections:
    - oracle.oci
  connection: local
  hosts: localhost
  vars:
    compartment_ocid: 'ocid1.compartment.oc1...'
    tenancy_namespace: 'frlzmy5hrkeg'
  tasks:
    - name: List bucket in Frankfurt region
      oci_object_storage_bucket_facts:
        config_profile_name: "DEFAULT"
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-frankfurt-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
        loop: "{{ result.buckets }}"

    - name: List bucket in Amsterdam region
      oci_object_storage_bucket_facts:
        config_profile_name: "DEFAULT"
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-amsterdam-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
        loop: "{{ result.buckets }}"
```

Provision an Oracle Database Software Image

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_software_image

```
---
- name: Create Database Software Image
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    compartment_ocid: "ocid1.compartment.oc1..."
    dbswimage_display_name: "PADBSWIMAGE2326"
    db_version: "23.0.0.0"
    image_shape: "EXADATA_SHAPE"
    db_image_type: "DATABASE_IMAGE"
    release_update: "23.26.2.0.0"
    one_offs: [] # e.g. one-off patch numbers

  tasks:
    - block:

        - name: Create DB Software Image
          oci_database_software_image:
            # required
            compartment_id: "{{ compartment_ocid }}"
            display_name: "{{ dbswimage_display_name }}"

            # optional
            database_version: "{{ db_version }}"
            image_shape_family: "{{ image_shape }}"
            image_type: "{{ db_image_type }}"
            patch_set: "{{ release_update }}"
            database_software_image_one_off_patches: "{{ one_offs }}"
            # ls_inventory: ""
            # source_db_home_id: ""

          register: dbswimage
        - debug:
            msg: "{{ dbswimage }}"
```

Provision an Oracle Home

- We can create an Oracle Home with 2 methods:
 - Deploy an Oracle Home
 - Create an Oracle Home at DB creation
- With both methods, we can just select the DB version which we have as default (currently 19c or Oracle AI Database 26ai — formerly 23ai, with RUs numbered 23.26.x) or create from a Database Custom Software Image
- If we want another PSU/RU than the latest one, we need to download it previously using dbasstools
- The recommended approach is to use Database Custom Software Images
- If you need to install custom one-offs, you can do it with traditional methods after Oracle Home deployment, or create a new OH from a Database Custom Software image with the one-offs and move the DB to the new OH

Provision an Oracle Home

- We can create an Oracle Home with 2 methods:
 - Deploy an Oracle Home
 - Create an Oracle Home at DB creation
- With both methods, we can just select the DB version which we have as default (currently 19c or Oracle AI Database 26ai — formerly 23ai, with RUs numbered 23.26.x) or create from a Database Custom Software Image
- If we want another PSU/RU than the latest one, we need to download it previously using dbasstools
- The recommended approach is to use Database Custom Software Images
- If you need to install custom one-offs, you can do it with traditional methods after Oracle Home deployment, or create a new OH from a Database Custom Software image with the one-offs and move the DB to the new OH

Provision an Oracle Home from Custom SW Image

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_db_home

```
---
- name: Create Oracle Home
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster.oc1... "
    database_software_image_ocid: "ocid1.databasesoftwareimage.oc1... "

  tasks:
    - block:

      - name: Create db_home with source = NONE
        oci_database_db_home:
          db_system_id: "{{ cloud_vmcluster_ocid }}"
          source: NONE
          state: present
          force_create: yes

          display_name: PADBHOMEANS
          database_software_image_id: "{{ database_software_image_ocid }}"
          # db_version: "23.0.0.0"

        register: dbhome

      - debug:
          msg: "{{ dbhome }}"
```

Using OCI Ansible Collection | Using the collection

Overruling modules arguments

- **Example playbook:**

```
---
- name: List summary of existing buckets in OCI object storage
  collections:
    - oracle.oci
  connection: local
  hosts: localhost
  vars:
    compartment_ocid: 'ocid1.compartment.oc1...'
    tenancy_namespace: 'frlzmy5hrkeg'
  tasks:
    - name: List bucket in Frankfurt region
      oci_object_storage_bucket_facts:
        config_profile_name: "DEFAULT"
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-frankfurt-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
        loop: "{{ result.buckets }}"

    - name: List bucket in Amsterdam region
      oci_object_storage_bucket_facts:
        config_profile_name: "DEFAULT"
        namespace_name: "{{ tenancy_namespace }}"
        compartment_id: "{{ compartment_ocid }}"
        region: "eu-amsterdam-1"
      register: result
    - name: Dump result
      debug:
        msg: 'Bucket name: {{item.name}}'
        loop: "{{ result.buckets }}"
```


Delete an Oracle Database Software Image

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_software_image

```
---
- name: Delete Database Software Image
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    database_software_image_ocid: "ocid1.databasesoftwareimage.oc1... "
  tasks:
    - block:
        - name: Delete DB Software Image
          oci_database_software_image:
            # required
            database_software_image_id: "{{ database_software_image_ocid }}"
            state: absent

          register: dbswimage
        - debug:
            msg: "{{ dbswimage }}"
```

Delete an Oracle Home

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_db_home

```
---
- name: Delete Oracle Home
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    db_home_ocid: "ocid1.dbhome.oc1... "

  tasks:
    - block:

      - name: Delete db_home
        oci_database_db_home:
          # required
          db_home_id: "{{ db_home_ocid }}"
          state: absent

          # optional
          perform_final_backup: false
          register: result
      - debug:
          msg: "{{ result }}"
```

Provision an OH and CDB using Custom SW Image

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_db_home

```
---
- name: Create Oracle Home & Database
  hosts: localhost
  collections: [ oracle.oci ]
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster.oc1... "

  tasks:
    - name: Create db_home with source = NONE
      oci_database_db_home:
        db_system_id: "{{ cloud_vmcluster_ocid }}"
        source: NONE
        state: present
        force_create: yes
        display_name: PADBHOMEANS
        database_software_image_id: "ocid1.databasesoftwareimage.oc1... "
        db_version: "23.0.0.0"
        database:
          admin_password: "{{ admin_password }}" # from OCI Vault
          db_name: PADBTANS
          db_unique_name: PADBTANS
          pdb_name: PADBTSTANSPDB1
          tde_wallet_password: "{{ tde_wallet_password }}"
          character_set: AL32UTF8
          ncharacter_set: AL16UTF16
          db_workload: OLTP
          sid_prefix: PADBTSTANS
        register: dbhome

# Full playbook (backup config, KMS keys, tags...):
# companion repo -> playbooks/provisioning/createOHCDB.yml
```

Provision an Oracle CDB

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_database

```
---
- name: Create Oracle Database
  hosts: localhost
  collections: [ oracle.oci ]
  connection: local
  vars:
    db_home_ocid: "ocid1.dbhome.oc1..."

  tasks:
    - name: Create database with source = NONE
      oci_database_database:
        db_home_id: "{{ db_home_ocid }}"
        state: present
        force_create: yes
        source: NONE
        db_version: "23.0.0.0"
        database:
          admin_password: "{{ admin_password }}" # from OCI Vault
          db_name: PADBANS1
          db_unique_name: PADBANS1
          pdb_name: PADBANS1PDB1
          tde_wallet_password: "{{ tde_wallet_password }}"
          character_set: AL32UTF8
          ncharacter_set: AL16UTF16
          db_workload: OLTP
          db_backup_config:
            auto_backup_enabled: true
            recovery_window_in_days: 56
            auto_backup_window: SLOT_ONE
          sid_prefix: PADBANS1
        register: database

# Full playbook (NFS backup destination, KMS keys, tags...):
# companion repo -> playbooks/provisioning/createCDB.yml
```

Provision an Oracle Non-CDB

Not available in OCI Ansible Collection

- We **can't** create Non-CDB Databases with the OCI Ansible Collection (as with the UI/other REST APIs). Note: the Non-CDB architecture is desupported starting with Oracle Database 21c — this only applies to 19c databases
- We need to adhere to a playbook build by connecting with SSH to any VM Cluster node and once there, to execute a DBaaS tool command
- Example:

```
---
- name: Creates a Non CDB
  hosts: exacs
  vars:
    db_name: "PADBANS1"
    oh_path: "/u02/app/oracle/product/19.0.0.0/dbhome_15"
    db_unique_name: "PADBANS1"
    pdb_name: "PADBANS1PDB1"
    character_set: "AL32UTF8"
    ncharacter_set: "AL16UTF16"
    admin_password: "WE1come12345##"
    tde_wallet_password: "WE1come12345##"

  tasks:
    - block:

        - name: Create Non-CDB
          become: yes
          become_method: sudo
          become_user: root
          command: dbaascli database create --oracleHome {{ oh_path }} --dbName {{ db_name }} --dbUniqueName {{ db_unique_name }} --pdbName {{
pdb_name }} --createAsCDB false --dbCharset {{ character_set }} --dbNCharset {{ ncharacter_set }}
          args:
            stdin: |
              {{ admin_password }}
              {{ admin_password }}
              {{ tde_wallet_password }}
              {{ tde_wallet_password }}
          register: result
        - debug:
            msg: "{{ result }}"
```

Delete an Oracle CDB

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_database

```
---
- name: Delete Oracle Database
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    database_ocid: "ocid1.database.oc1... "

  tasks:
    - block:

      - name: Delete database
        oci_database_database:
          # required
          database_id: "{{ database_ocid }}"
          state: absent

          # optional
          perform_final_backup: false
          register: result

      - debug:
          msg: "{{ result }}"
```

Provision an Oracle PDB

- Pluggable operations are supported only for 19c or later databases
- You can just create an Oracle PDB at the same time (not parallel creation is allowed)
- During PDB creation the parent database (CDB) is in “Updating” state
- Database would be created in all the nodes of the VM Cluster
- Create/adjust GI services to have single/multi instances running PDBs

Provision an Oracle PDB

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_pluggable_database

```
---
- name: Create Pluggable Database
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    container_database_ocid: "ocid1.database.oc1..."
    # compartment_ocid: "ocid1.compartment.oc1..."

  tasks:
    - block:

      - name: Create pluggable database
        oci_database_pluggable_database:
          # required
          pdb_name: "PADBANS2PDB2"
          container_database_id: "{{ container_database_ocid }}"
          state: present
          force_create: yes

          # optional
          # compartment_id: "{{ compartment_ocid }}"
          pdb_admin_password: "WElcome12345##"
          tde_wallet_password: "WElcome12345##"
          should_pdb_admin_account_be_locked: false
          # freeform_tags: {'Department': 'Finance'}
          # defined_tags: {'Operations': {'CostCenter': 'US'}}

        register: pdb
      - debug:
          msg: "{{ pdb }}"
```


Delete an Oracle PDB

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_pluggable_database

```
---
- name: Delete Oracle Pluggable Database
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    pluggable_database_ocid: "ocid1.pluggabledatabase.oc1.eu-frankfurt-1.antheljrwwun52iao45i3e4uavmpjw6nc6p5rnvtq66h3r42xt7l3n4x4bna"

  tasks:
    - block:

      - name: Delete pluggable database
        oci_database_pluggable_database:
          # required
          pluggable_database_id: "{{ pluggable_database_ocid }}"
          state: absent

        register: result
      - debug:
          msg: "{{ result }}"
```

Provision an Oracle Data Guard Association

- Requires two existing Exadata VM Clusters: one containing an existing DB and one for the new standby database
- Oracle strongly recommends the primary and standby DBs for any production workloads be on different ExaCS Infrastructures for better fault isolation and disaster protection
- You can create or reuse an Oracle Home on the standby Exadata Instance. ExaDB-D now supports different RUs for primary and standby DB Homes (also in switchover and failover operations), although keeping them aligned via Database Software Images is still the recommended practice
- If you are planning to use customer-managed keys to encrypt the database, you must have configured the OCI Vault Service
- If the DG is in the same region. The primary and standby databases must be part of the same VCN
- Oracle recommends that the instance containing the standby DB be in a different availability domain from the instance containing the primary DB to improve availability and disaster recovery
- If you want to configure Oracle Data Guard across regions, then you must configure remote virtual cloud network (VCN) peering between the primary and standby databases
- Configure the ingress and egress security rules for the subnets of both Exadata Cloud Infrastructure instances in the Oracle Data Guard association to enable TCP traffic to move between the applicable ports.



Provision an Oracle Data Guard Association reusing an existing VM

Cluster OCI Ansible Collection

- **Module:** oracle.oci.oci_database_data_guard_association
- Valid combinations of transport/protection_mode:
 - MAXIMUM_PERFORMANCE – ASYNC or MAXIMUM_AVAILABILITY - SYNC

```
---
- name: Creates an Oracle Data Guard Association in an existing VM Cluster
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    peer_vm_cluster_ocid: "ocid1.cloudvmcluster.oc1..."

  tasks:
    - block:

      - name: Creates DG association for a DB in an existing VM Cluster
        oci_database_data_guard_association:
          # required
          creation_type: ExistingVmCluster
          database_admin_password: "WELcome12345##"
          protection_mode: MAXIMUM_AVAILABILITY
          transport_type: SYNC
          state: present

          # optional
          peer_vm_cluster_id: "{{ peer_vm_cluster_ocid }}"
          database_software_image_id: "ocid1.databasesoftwareimage.oc1..."
          peer_db_unique_name: PADBANSSTB
          peer_sid_prefix: PADBANSSTB
          peer_db_home_id: "ocid1.dbhome.oc1..."
          is_active_data_guard_enabled: true

        register: result
      - debug:
          msg: "{{ result }}"
```

Provision an Oracle Data Guard Association creating a new VM Cluster

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_data_guard_association
- Valid combinations of transport/protection_mode:
 - MAXIMUM_PERFORMANCE – ASYNC or MAXIMUM_AVAILABILITY - SYNC

```
---
- name: Create DG Association creating a New VM Cluster
  hosts: localhost
  collections: [ oracle.oci ]
  connection: local

  tasks:
    - name: Create DG association (new VM Cluster)
      oci_database_data_guard_association:
        creation_type: NewDbSystem
        database_admin_password: "{{ admin_password }}"
        protection_mode: MAXIMUM_AVAILABILITY
        transport_type: SYNC
        state: present
        display_name: VMCluster2
        availability_domain: "bWnP:EU-FRANKFURT-1-AD-2"
        cpu_core_count: 4
        subnet_id: "ocid1.subnet.oc1..xxxxxxEXAMPLExxxxxx"
        hostname: emeaexacsvm2
        license_model: BRING_YOUR_OWN_LICENSE
        database_software_image_id: "ocid1.databasesoftwareimage.oc1..."
        peer_db_unique_name: PADBANSSTB
        peer_sid_prefix: PADBANSSTB
        is_active_data_guard_enabled: true
        register: result

# Full playbook (NSGs, time zone, data collection opts...):
# companion repo -> playbooks/provisioning/createDGNewVMC.yml
```

Demo | Create a DB SW Image, OH, CDB & PDB

PROPOSED DEMO

❑ 1) Create a DB Software Image:

```
$ ansible-playbook createDBSWImage.yml
```

❑ 2) Create an Oracle DB Home:

```
$ ansible-playbook createOH.yml
```

❑ 3) Create an Oracle CDB:

```
$ ansible-playbook createCDB.yml
```

❑ 4) Create an Oracle PDB:

```
$ ansible-playbook createPDB.yml
```

NOTE: git clone the workshop companion repository and replace the needed OCIDs

Change Operations Scenarios

Change Operations Scenarios

- In this section we'll talk about the following Change Operations Scenarios, such as:
 - Starting/stopping/modifying a VM Cluster
 - Starting/Stopping/Restarting a VM Cluster node
 - Starting/Stopping/Restarting a database
 - Starting/Stopping/Restarting a PDB
 - Perform a Data Guard Switchover
 - Perform a Data Guard Failover
 - Perform a Data Guard re-instantiation

Starting/stopping/modifying a VM Cluster

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_cloud_vm_cluster
- Play with the parameters ***cpu_core_count*** or ***ocpu_count*** (mutually exclusive) to stop (0) or start (any value) the whole VM Cluster gracefully. On X11M infrastructure, CPU is expressed in ECPU's. You can use the same playbook to modify other VM Cluster parameters

```
---
- name: Scale ExaCS VM Cluster
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster..."

  tasks:
    - block:

      - name: Update cloud_vm_cluster
        oci_database_cloud_vm_cluster:
          # required
          cloud_vm_cluster_id: "{{cloud_vmcluster_ocid }}"
          # optional
          # display_name: "EmeaExaCS_VMCluster1"
          cpu_core_count: 0
          # ocpu_count: 3.4 # <- For fractional
          # memory_size_in_gbs: 56
          # db_node_storage_size_in_gbs: 56
          # data_storage_size_in_tbs: 3.4
          # storage_size_in_gbs: 56

      (...)
    
```

```
# More optional parameters:
#   memory_size_in_gbs
#   data_storage_size_in_tbs
#   db_node_storage_size_in_gbs
#   license_model, ssh_public_keys
#   nsg_ids, backup_network_nsg_ids
#   freeform_tags / defined_tags
#   update_details (see Maintenance)
#   data_collection_options

# Full parameter list:
#   companion_repo ->
#   playbooks/change/scaleVMC.yml

register: vmc

- debug:
    msg: "{{ vmc }}"
  
```


Starting/Stopping/Restarting a VM Cluster node

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_db_node_actions
- Play with the parameter ***action*** (stop/start/softreset/reset)

```
---
- name: Restart an ExaCS VM Cluster DB Node
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    db_node_ocid: "ocid1.dbnode.oc1... "

  tasks:
    - block:

      - name: Stop VM Cluster DB Node
        oci_database_db_node_actions:
          # required
          db_node_id: "{{ db_node_ocid }}"
          action: stop

        register: dbnode
      - debug:
          msg: "{{ dbnode }}"

      - name: Start VM Cluster DB Node
        oci_database_db_node_actions:
          # required
          db_node_id: "{{ db_node_ocid }}"
          action: start

        register: dbnode
      - debug:
          msg: "{{ dbnode }}"
```

Starting/Stopping a database

Not available in OCI Ansible Collection

- We ***can't*** stop/start Databases (CDB/Non-CDB) with the OCI Ansible Collection (as with the UI/other REST APIs)
- We need to adhere to a playbook build by connecting with SSH to any VM Cluster node and once there, to execute a DBaaS tool command, or just the GI DB management command (srvctl)
- Example:

```
---
- name: Restart an Oracle DB running in ExaCS
  hosts: exacs
  vars:
    db_name: "PADBTST1"

  tasks:
    - block:

      - name: Stop DB
        become: yes
        become_method: sudo
        become_user: root
        shell: dbaascli database stop --dbname {{ db_name }}
        register: database

      - debug:
          msg: "{{ database }}"

      - name: Start DB
        become: yes
        become_method: sudo
        become_user: root
        shell: dbaascli database start --dbname {{ db_name }}
        register: database

      - debug:
          msg: "{{ database }}"
```

Starting/Stopping/Restarting a PDB

OCI Ansible Collection

- **Module:** oracle.oci.oci database pluggable database actions
- Play with the parameter ***action*** (stop/start)

```
---
- name: Restart an Oracle PDB
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    pluggable_database_ocid: "ocid1.pluggabledatabase.oc1... "

  tasks:
    - block:

      - name: Stop a PDB
        oci_database_pluggable_database_actions:
          # required
          pluggable_database_id: "{{ pluggable_database_ocid }}"
          action: stop

        register: pdb
      - debug:
          msg: "{{ pdb }}"

      - name: Start a PDB
        oci_database_pluggable_database_actions:
          # required
          pluggable_database_id: "{{ pluggable_database_ocid }}"
          action: start

        register: pdb
      - debug:
          msg: "{{ pdb }}"
```

Perform a Data Guard Switchover

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_data_guard_association_actions

```
---
- name: Perform Oracle Data Guard switchover operation
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    # required
    database_ocid: "ocid1.database.oc1..."
    data_guard_association_ocid: "ocid1.dataguardassociation.oc1..."
    database_admin_pw: "WElcome12345##"

  tasks:
    - block:

        - name: Performs Data Guard switchover operation
          oci_database_data_guard_association_actions:
            # required
            database_id: "{{ database_ocid }}"
            data_guard_association_id: "{{ data_guard_association_ocid }}"
            database_admin_password: "{{ database_admin_pw }}"
            action: switchover

          register: result
        - debug:
            msg: "{{ result }}"
```

Perform a Data Guard Failover

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_data_guard_association_actions

```
---
- name: Perform Oracle Data Guard failover operation
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    # required
    database_ocid: "ocid1.database.oc1..."
    data_guard_association_ocid: "ocid1.dataguardassociation.oc1..."
    database_admin_pw: "WElcome12345##"

  tasks:
    - block:

        - name: Performs Data Guard failover operation
          oci_database_data_guard_association_actions:
            # required
            database_id: "{{ database_ocid }}"
            data_guard_association_id: "{{ data_guard_association_ocid }}"
            database_admin_password: "{{ database_admin_pw }}"
            action: failover

          register: result
        - debug:
            msg: "{{ result }}"
```

Perform a Data Guard Re-instantiation

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_data_guard_association_actions

```
---
- name: Perform Oracle Data Guard reinstate operation
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    # required
    database_ocid: "ocid1.database.oc1..."
    data_guard_association_ocid: "ocid1.dataguardassociation.oc1..."
    database_admin_pw: "WElcome12345##"

  tasks:
    - block:

        - name: Performs Data Guard reinstate operation
          oci_database_data_guard_association_actions:
            # required
            database_id: "{{ database_ocid }}"
            data_guard_association_id: "{{ data_guard_association_ocid }}"
            database_admin_password: "{{ database_admin_pw }}"
            action: reinstate

          register: result
        - debug:
            msg: "{{ result }}"
```

Demo | Using OCI Ansible Collection to scale up/down OCPUs

PROPOSED DEMO

- ❑ 1) Scale up the VM Cluster
`$ ansible-playbook scaleVMC.yml`
- ❑ 2) Scale down VM Cluster OCPU:
`$ ansible-playbook scaleVMC.yml`
- ❑ 3) Restart VM Cluster node:
`$ ansible-playbook restartDBNode.yml`
- ❑ 4) Restart a CDB:
`$ ansible-playbook restartCDB.yml`
- ❑ 5) Restart a PDB:
`$ ansible-playbook restartPDB.yml`

NOTE: git clone the workshop companion repository and replace the needed OCIDs

Maintenance Operations Scenarios

Maintenance Operations Scenarios

- In this section we'll talk about the following Maintenance Operations Scenarios, such as:
 - OS patching
 - GI patching
 - DBs patching

domU OS/Exadata software patching

- Customer is responsible for OS, GI and DB Software Updates to maintain stability and security compliance. Guest VMs currently run Oracle Linux 8 with Exadata System Software 24.x/25.x
- Zero database downtime with RAC rolling and Application Continuity
- During window, prepare for reduced DB compute processing
- GI-certified version should be reviewed in “Exadata Cloud Service Software Versions (Doc ID 2333222.1)”
- Beware of the OEM and plug-ins versions compatibility before the Exadata software patching (Doc ID 1626579.1)
- It is possible that many 3rd party packages/software (also known as custom packages) installed from you, could be deleted in order to satisfy Linux dependencies. You would have to install new versions according to the target Linux versions. Software that tainted kernel is usually forbidden. Beware of zipped distributed software also as it can stop working
- In a 2 node cluster, first node is patched from the second one and after it is patched successfully, second one is patched from the first
- Prechecks or apply logs can be found at:
 /u02/dbserver.patch.zip_exadata_ol8_<version>_Linux-x86-64.zip/dbserver_patch_<release date>/patchmgr_log...
 /u02/cellos/dbnodeupdate.log



Manage ExaCS Infra Patching Schedule

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_cloud_exadata_infrastructure
Exadata Infrastructure quarterly maintenance is Oracle-managed: you control WHEN it runs.
lead_time_in_weeks sets how far in advance Oracle schedules it

```
---
- name: Manage ExaCS Infra quarterly maintenance
  hosts: localhost
  collections: [ oracle.oci ]
  connection: local
  vars:
    exainfra_ocid: "ocid1.cloudexadatainfrastructure..."
  tasks:
    - name: Set maintenance window (patching schedule)
      oci_database_cloud_exadata_infrastructure:
        cloud_exadata_infrastructure_id: "{{ exainfra_ocid }}"
        maintenance_window:
          preference: CUSTOM_PREFERENCE
          patching_mode: ROLLING
          months:
            - name: FEBRUARY
            - name: MAY
            - name: AUGUST
            - name: NOVEMBER
          weeks_of_month: [ 2 ]
          days_of_week:
            - name: SATURDAY
          hours_of_day: [ 2 ]
          lead_time_in_weeks: 4
        register: infra
    - debug:
        msg: "{{ infra.cloud_exadata_infrastructure.maintenance_window }}"
```

List OS Updates

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_cloud_vm_cluster_update_facts

```
---
- name: List ExaCS VM Cluster OS updates
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster.oc1... "

  tasks:
    - block:

      - name: List cloud_vm_cluster OS updates
        oci_database_cloud_vm_cluster_update_facts:
          # required
          cloud_vm_cluster_id: "{{ cloud_vmcluster_ocid }}"

          # optional
          update_type: OS_UPDATE

        register: vmc_update
      - debug:
          msg: "{{ vmc_update }}"
```

List GI Updates

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_cloud_vm_cluster_update_facts

```
---
- name: List ExaCS VM Cluster GI patches
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster.oc1... "

  tasks:
    - block:

      - name: List cloud_vm_cluster GI patches
        oci_database_cloud_vm_cluster_update_facts:
          # required
          cloud_vm_cluster_id: "{{ cloud_vmcluster_ocid }}"

          # optional
          update_type: GI_PATCH

        register: vmc_update
      - debug:
          msg: "{{ vmc_update }}"
```

OS/GI Patch Precheck/Rolling Apply

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_cloud_vm_cluster
- ***update_action*** can be: PRECHECK / ROLLING_APPLY

```
---
- name: Precheck update ExaCS VM Cluster (GI or OS, depends on update_id)
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster.oc1... "
    update_ocid: "ocid1.dbupdate.oc1... "

  tasks:
    - block:

      - name: Precheck Update cloud_vm_cluster
        oci_database_cloud_vm_cluster:
          # required
          cloud_vm_cluster_id: "{{ cloud_vmcluster_ocid }}"

          # optional
          update_details:
            # optional
            update_id: "{{ update_ocid }}"
            update_action: PRECHECK

        register: vmc
      - debug:
          msg: "{{ vmc }}"
```

Error handling in long-running operations

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_cloud_vm_cluster

ExaCS maintenance operations run for hours (GI/OS patching): combine block / rescue / always, raise wait_timeout on the module and command_timeout in ansible.cfg

```
---
- name: GI rolling apply with error handling
  hosts: localhost
  collections: [ oracle.oci ]
  connection: local
  vars:
    cloud_vmcluster_ocid: "ocid1.cloudvmcluster... "
    update_ocid: "ocid1.dbupdate.oc1... "
  tasks:
    - block:
      - name: Apply GI update (rolling)
        oci_database_cloud_vm_cluster:
          cloud_vm_cluster_id: "{{ cloud_vmcluster_ocid }}"
          update_details:
            update_id: "{{ update_ocid }}"
            update_action: ROLLING_APPLY
          wait_timeout: 14400    # 4h
          register: vmc
      - rescue:
        - name: Collect diagnostics on failure
          oci_database_cloud_vm_cluster_facts:
            cloud_vm_cluster_id: "{{ cloud_vmcluster_ocid }}"
          register: st
        - fail:
          msg: "Failed: {{ st.cloud_vm_clusters[0].lifecycle_state }}"
      - always:
        - debug:
          msg: "Finished at {{ ansible_date_time.iso8601 }}"
```

GI patching

- Patching the GI with the OCI Ansible collection follows the same process that from the UI/OCI CLI
- You can't influence in the process and the cluster will be patched in a rolling fashion
- By using dbaascli, you can just apply the patching to a subset of nodes or use GI Golden Images
- Logs of the tool can be seen in:
 - `/var/opt/oracle/log/gridPatch/gridPatch.log`

Databases patching

- We have 2 possibilities:
 - **In-Place** patching: Patching the OH binaries with a new RU version or with a Custom SW Image
 - **Out-of-Place** patching: Move a DB to a new home with a higher version and with optionally additional one-offs installed (this is the Oracle recommended approach)
- For the rollback:
 - In-Place rollback: Uninstall manually any one-off and rollback the OH update
 - Out-of-Place rollback: Just move the DB to the original OH
- Patching the DB with the OCI Ansible collection follows the same process that from the UI/OCI CLI (in-place or out-of –place)
- Any kind of patching is performed in a rolling fashion. First DB instance patched/moved, then second node...
- The Datapatch is applied once last node has been patched. In a Data Guard standby environment, this is detected and the Data Patch is not applied
- By using the OCI Ansible Collection modules you can't influence the patching process (apply on a subset of nodes or move different DBs in parallel). For this, try to use the DBaaS tool command even in a custom Ansible Playbook
- Logs:
 - In-place: /var/opt/oracle/log/dbHomePatch/dbHomePatch.log
 - Out-of-place: /var/opt/oracle/log/<DBNAME>/database/move/pilot_<date>



DB In-Place Patch Precheck/Apply

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_db_home
- An already existing DB Custom SW image can be used or a patch_id or a known version
- Possible ***action*** values are: PRECHECK / APPLY

```
---
- name: Patch In-Place Oracle Databases by patching OH
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    db_home_ocid: "ocid1.dbhome.oc1... "
    target_database_software_image_id: "ocid1.databasesoftwareimage.oc1... "

  tasks:
    - block:

      - name: Patch Databases in OH
        oci_database_db_home:
          # required
          db_home_id: "{{ db_home_ocid }}"

          # optional
          patch_details:
            # optional
            # patch_id: "ocid1.patch.oc1..xxxxxxEXAMPLExxxxxx"
            database_software_image_id: "{{ target_database_software_image_id }}"
            action: PRECHECK
            # one_off_patches: [ "one_off_patches_example" ]

        register: result
      - debug:
          msg: "{{ result }}"
```

DB Out-of-Place Patch Precheck/Apply

OCI Ansible Collection

- **Module:** oracle.oci.oci_database_db_home
- Target DB Home needs to be deployed upfront
- This performs an Oracle Database move operations without any precheck

```
---
- name: Patch Oracle Database out-of-place
  hosts: localhost
  collections:
    - oracle.oci
  connection: local
  vars:
    database_ocid: "ocid1.database.oc1... "
    target_db_home_ocid: "ocid1.dbhome.oc1... "

  tasks:
    - block:

      - name: Patch database by moving to a new Oracle Home
        oci_database_database:
          # required
          database_id: "{{ database_ocid }}"
          db_home_id: "{{ target_db_home_ocid }}"

        register: database

      - debug:
          msg: "{{ database }}"
```

Demo | Perform GI/DB Patch precheck

PROPOSED DEMO

❑ 1) OS Patching:

- List available patches for OS:

```
$ ansible-playbook listVMCOSupdates.yml
```

- OS patch precheck:

```
$ ansible-playbook patchVMCOSGI-precheck
```

❑ 2) GI Patching:

- List available patches for GI:

```
$ ansible-playbook listVMCOSupdates.yml
```

- GI patch precheck:

```
$ ansible-playbook patchVMCOSGI-precheck
```

❑ 3) DB in-place:

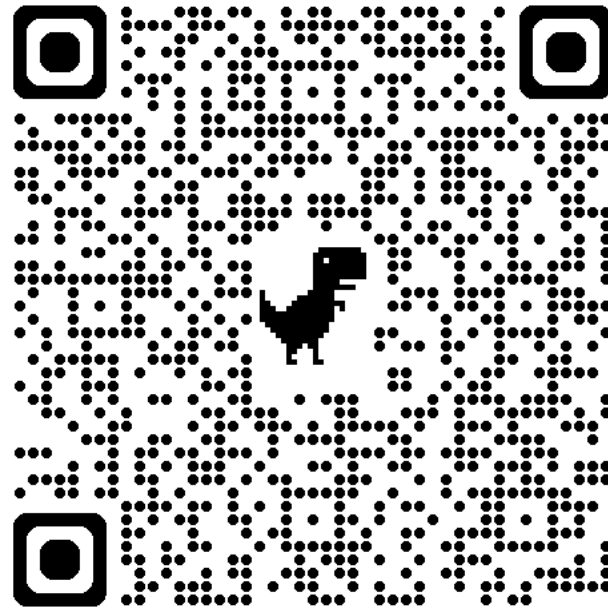
- DB in-place patch precheck (using DB SW Image):

```
$ ansible-playbook patchInPlaceCDB-precheck.yml
```

❑ 4) DB out-of-place:

```
$ ansible-playbook patchOutOfPlaceCDB.yml
```

Stay tuned



Operations Advisory
GitHub Repository



Thank you

ORACLE